



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

FACULTAD DE INGENIERÍA

AREA MECÁNICA ELÉCTRICA



PROYECTO PARCIAL 2

TRATAMIENTO DE IMÁGENES

ALUMNO: RAMÍREZ GALVÁN MANUEL

CLAVE: 288093

PROFESOR: JULLY KADO MERCADO ELIAS

FECHA: 10/MARZO/2025

Contenido

Introducción	3
Desarrollo	3
Explicación del Código	3
Código Completo	6
Imágenes Utilizadas	7
Resultados	7
Conclusiones	7
Referencias	8
Ilustración 1	3
Ilustración 2	3
Ilustración 3	3
Ilustración 4	4
Ilustración 5	4
Ilustración 6	4
Ilustración 7	4
Ilustración 8	5
Ilustración 9	5
Ilustración 10	5
Ilustración 11	5
Ilustración 12	5
Ilustración 13 Código Completo	6
Ilustración 14 Imágenes Utilizadas	7
Ilustración 15 Resultados Finales	7

Introducción

El análisis de imágenes médicas es una herramienta muy útil en el diagnóstico y tratamiento de diversas enfermedades, incluyendo el cáncer. En este contexto, la detección automática de tumores en tomografías computarizadas representa un gran avance en la asistencia a médicos radiólogos, permitiendo una identificación más rápida y precisa de anomalías.

Este proyecto se enfoca en la implementación de técnicas de procesamiento digital de imágenes para la detección de tumores cerebrales en tomografías. Utilizando la librería OpenCV y técnicas de segmentación de imágenes, el objetivo es resaltar las áreas sospechosas que podrían corresponder a tumores, mejorando así la visualización y el análisis de las imágenes médicas.

Desarrollo

Explicación del Código

Se coloca la librería de OpenCV, NumPy y Matplotlib para trabajar con ellas en manipulación y procesamiento de las imágenes, operaciones con números y matrices de imágenes e impresión adecuada de las imágenes en ventanas.

```
1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
```

Ilustración 1

Se carga la primera imagen en escala de grises, esto ya que la segmentación de los bordes es más precisa en imágenes de un solo canal, que es en blanco y negro.

```
5 # Cargar la primera imagen en escala de grises
6 image1_path = "t1.jpg"
7 image1 = cv2.imread(image1_path, cv2.IMREAD_GRAYSCALE)
```

Ilustración 2

Se reduce el ruido con un Filtro Gaussiano, se utiliza un tamaño (5,5) ya que es suficiente para reducir el ruido sin difuminarse tanto los bordes para perder detalles del mismo.

```
9 # Aplicar filtro Gaussiano para reducción de ruido
10 image1_blur = cv2.GaussianBlur(image1, (5, 5), 0)
```

Ilustración 3

Se utiliza la corrección de contraste con CLAHE (Contrast Limited Adaptive Histogram Equalization) lo cual ajusta el contraste de manera localizada y ayuda a resaltar mejor las zonas pequeñas sin afectar otras partes de la imagen. Se podía utilizar también la ecualización estándar (cv2.equalizeHist()), pero da mejores resultados así.

El clipLimit tiene un valor de 3 ya que mejora el contraste sin hacer una gran sobre exposición, ya que al aumentarlo resaltaba aun mas zonas como el contorno del cráneo o partes blancas como las líneas más anchas del cerebro

Se utilizo un tileGridSize (5,5) para se ajustará aún mejor y resaltará mejor el tumor, ya que al disminuirlo tomaba más área alrededor del tumor y al aumentarlo tomaba menor área del tumor.

```
12 # Aplicar corrección de contraste adaptativa CLAHE
13 clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(5,5))
14 image1_clahe = clahe.apply(image1_blur)
```

Ilustración 4

Se hace una umbralización para la segmentación, se un valor de 140 ya que al disminuir el umbral hay mas ruido y toma más partes además del tumor y al aumentar el umbral se pierde el tumor y se ve todo negro; además este valor este intermedio ya que el tumor esta en una escala de grises y no está muy claro o llegando a los blancos.

```
16 # Aplicar umbralización manual con un umbral específico
17 threshold_value1 = 140
18 _, thresh1 = cv2.threshold(image1_clahe, threshold_value1, 255, cv2.THRESH_BINARY)
```

Ilustración 5

Se utiliza la operación morfológica de Apertura que se busca eliminar pequeños ruidos blancos en la imagen sin afectar el tamaño del tumor segmentado, además de no rellenar el mismo tumor u otras zonas como lo hará el Cierre.

Se utiliza un kernel de (8,8) ya que mas pequeño aun queda mucho ruido y toma en consideración pequeñas zonas y más grande toma un área menor a la del tumor hasta llegar a quitar todo el tumor y se hacen 2 iteraciones porque con una aún quedan zonas blancas y no solo el tumor.

```
20 # Aplicar operaciones morfológicas
21 kernel = np.ones((8, 8), np.uint8)
22 morph1 = cv2.morphologyEx(thresh1, cv2.MORPH_OPEN, kernel, iterations=2)
```

Ilustración 6

Se hace la detección de los contornos con la aproximación simple y se dibujan de color azul con un grosor de 2.

```
24 # Encontrar contornos
25 contours1, _ = cv2.findContours(morph1, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
26 # Dibujar contornos
27 image1_contours = cv2.cvtColor(image1, cv2.COLOR_GRAY2BGR)
28 cv2.drawContours(image1_contours, contours1, -1, (255, 0, 0), 2)
```

Ilustración 7

Se carga la segunda imagen en escala de grises, se utilizan los mismos valores para las operaciones como el Filtro Gaussiano y la corrección de contraste CLAHE.

```

30 # Cargar la segunda imagen en escala de grises
31 image2_path = "t2.jpg"
32 image2 = cv2.imread(image2_path, cv2.IMREAD_GRAYSCALE)
33
34 # Aplicar filtro Gaussiano para reducción de ruido
35 image2_blur = cv2.GaussianBlur(image2, (5, 5), 0)
36
37 # Aplicar corrección de contraste adaptativa CLAHE
38 image2_clahe = clahe.apply(image2_blur)

```

Ilustración 8

En la parte de la umbralización se necesitó un umbral más alto ya que esta segunda imagen tiene mas brillo y necesita un umbral más alto, por eso su valor es 240.

```

40 # Aplicar umbralización manual con un umbral específico
41 threshold_value2 = 240
42 _, thresh2 = cv2.threshold(image2_clahe, threshold_value2, 255, cv2.THRESH_BINARY)

```

Ilustración 9

Se utilizo igualmente la Apertura con el mismo valor del kernel y con 2 iteraciones para tener mejores resultados tambien.

```

44 # Aplicar operaciones morfológicas
45 morph2 = cv2.morphologyEx(thresh2, cv2.MORPH_OPEN, kernel, iterations=2)

```

Ilustración 10

Se hace la detección y dibujo de contornos del mismo color y grosor.

```

47 # Encontrar contornos
48 contours2, _ = cv2.findContours(morph2, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
49 # Dibujar contornos
50 image2_contours = cv2.cvtColor(image2, cv2.COLOR_GRAY2BGR)
51 cv2.drawContours(image2_contours, contours2, -1, (255, 0, 0), 2)

```

Ilustración 11

Visualización de las imágenes en ventana acomodadas por Matplotlib.

```

53 # Mostrar resultados
54 fig, axs = plt.subplots(2, 3, figsize=(20, 10))
55
56 axs[0, 0].imshow(image1, cmap='gray')
57 axs[0, 0].set_title("Imagen 1 Original")
58 axs[0, 0].axis("off")
59 axs[0, 1].imshow(morph1, cmap='gray')
60 axs[0, 1].set_title("Segmentación del Tumor 1")
61 axs[0, 1].axis("off")
62 axs[0, 2].imshow(image1_contours)
63 axs[0, 2].set_title("Detección de Tumor 1")
64 axs[0, 2].axis("off")
65
66 axs[1, 0].imshow(image2, cmap='gray')
67 axs[1, 0].set_title("Imagen 2 Original")
68 axs[1, 0].axis("off")
69 axs[1, 1].imshow(morph2, cmap='gray')
70 axs[1, 1].set_title("Segmentación del Tumor 2")
71 axs[1, 1].axis("off")
72 axs[1, 2].imshow(image2_contours)
73 axs[1, 2].set_title("Detección de Tumor 2")
74 axs[1, 2].axis("off")
75
76 plt.show()

```

Ilustración 12

Código Completo

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Cargar la primera imagen en escala de grises
image1_path = "t1.jpg"
image1 = cv2.imread(image1_path, cv2.IMREAD_GRAYSCALE)

# Aplicar filtro Gaussiano para reducción de ruido
image1_blur = cv2.GaussianBlur(image1, (5, 5), 0)

# Aplicar corrección de contraste adaptativa CLAHE
clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(5,5))
image1_clahe = clahe.apply(image1_blur)

# Aplicar umbralización manual con un umbral específico
threshold_value1 = 140
_, thresh1 = cv2.threshold(image1_clahe, threshold_value1, 255, cv2.THRESH_BINARY)

# Aplicar operaciones morfológicas
kernel = np.ones((8, 8), np.uint8)
morph1 = cv2.morphologyEx(thresh1, cv2.MORPH_OPEN, kernel, iterations=2)

# Encontrar contornos
contours1, _ = cv2.findContours(morph1, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
image1_contours = cv2.cvtColor(image1, cv2.COLOR_GRAY2BGR)
cv2.drawContours(image1_contours, contours1, -1, (255, 0, 0), 2)

# Cargar la segunda imagen en escala de grises
image2_path = "t2.jpg"
image2 = cv2.imread(image2_path, cv2.IMREAD_GRAYSCALE)

# Aplicar filtro Gaussiano para reducción de ruido
image2_blur = cv2.GaussianBlur(image2, (5, 5), 0)

# Aplicar corrección de contraste adaptativa CLAHE
image2_clahe = clahe.apply(image2_blur)

# Aplicar umbralización manual con un umbral específico
threshold_value2 = 240
_, thresh2 = cv2.threshold(image2_clahe, threshold_value2, 255, cv2.THRESH_BINARY)

# Aplicar operaciones morfológicas
morph2 = cv2.morphologyEx(thresh2, cv2.MORPH_OPEN, kernel, iterations=2)

# Encontrar contornos
contours2, _ = cv2.findContours(morph2, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
image2_contours = cv2.cvtColor(image2, cv2.COLOR_GRAY2BGR)
cv2.drawContours(image2_contours, contours2, -1, (255, 0, 0), 2)

# Mostrar resultados
fig, axs = plt.subplots(2, 3, figsize=(20, 10))

axs[0, 0].imshow(image1, cmap='gray')
axs[0, 0].set_title("Imagen 1 Original")
axs[0, 0].axis("off")
axs[0, 1].imshow(morph1, cmap='gray')
axs[0, 1].set_title("Segmentación del Tumor 1")
axs[0, 1].axis("off")
axs[0, 2].imshow(image1_contours)
axs[0, 2].set_title("Detección de Tumor 1")
axs[0, 2].axis("off")

axs[1, 0].imshow(image2, cmap='gray')
axs[1, 0].set_title("Imagen 2 Original")
axs[1, 0].axis("off")
axs[1, 1].imshow(morph2, cmap='gray')
axs[1, 1].set_title("Segmentación del Tumor 2")
axs[1, 1].axis("off")
axs[1, 2].imshow(image2_contours)
axs[1, 2].set_title("Detección de Tumor 2")
axs[1, 2].axis("off")

plt.show()
```

Ilustración 13 Código Completo

Imágenes Utilizadas

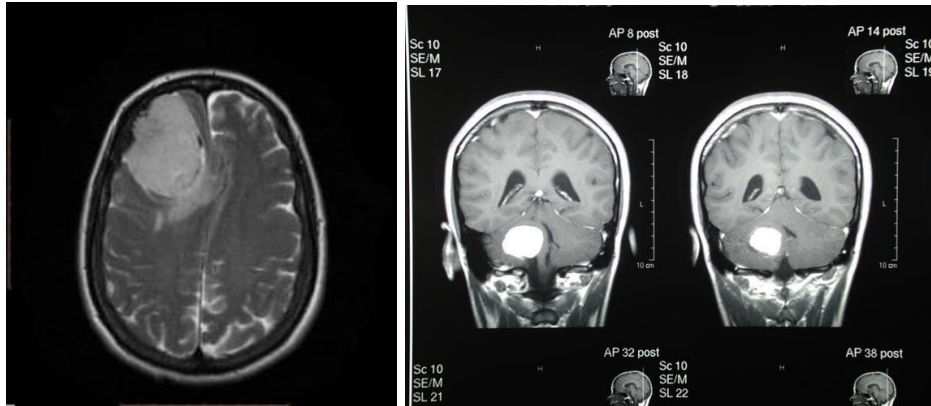


Ilustración 14 Imágenes Utilizadas

Resultados

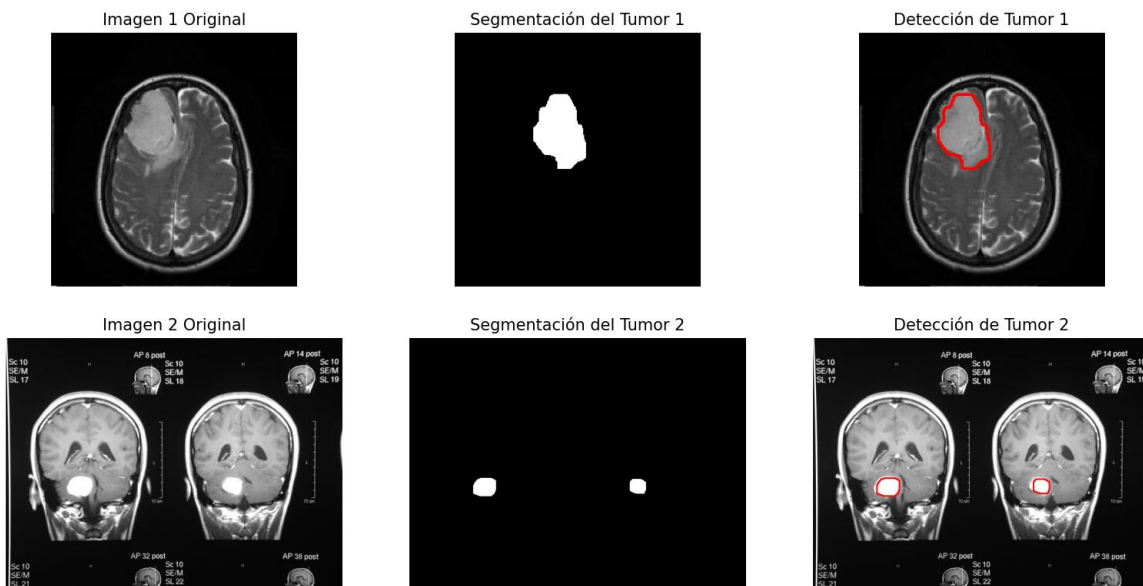


Ilustración 15 Resultados Finales

Conclusiones

El utiliza el procesamiento digital de imágenes para la detección de tumores en tomografías computarizadas. A través del uso de técnicas como filtrado Gaussiano, ecualización de contraste con CLAHE, umbralización de forma manual, operaciones morfológicas y detección de contornos, se logró resaltar y segmentar las posibles regiones tumorales.

Se logró la identificación del tumor en diferentes imágenes, ajustando solo el parámetro de umbralización según el brillo y contraste de cada tomografía. Se vio que la elección adecuada de umbrales y filtros es importante para evitar la detección de regiones no tumorales y detectar los que, si son, esto a prueba y error en la mayoría de los casos.

Referencias

OpenCV: Histograms. (2025). Recuperado el 5 de marzo de 2025, del sitio web de Opencv.org:

https://docs.opencv.org/4.x/d6/dc7/group_imgproc_hist.html#gad3b7f72da85b821fda2bc41687573974

CLAHE Histogram Equalization – OpenCV. (2023). Recuperado el 5 de marzo de 2025, del sitio web de Tutorialspoint.com: <https://www.tutorialspoint.com/clahe-histogram-equalization-ndash-opencv>

Ramírez Galván, M. (2025). Actividades de procesamiento de imágenes mediante técnicas de filtrado y segmentación (P2_A1, P2_A2, P2_A3, P2_A4). Universidad Autónoma de San Luis Potosí.

ChatGPT